

可以把这段演化分成三层：

Gated DeltaNet  $\rightarrow$  KDA  $\rightarrow$  Kimi K3 混合 Attention

它们分别解决三个不同问题：

1. **Gated DeltaNet**: 整个状态应该遗忘多少？
2. **KDA**: 状态中的不同通道，应该分别遗忘多少？
3. **Kimi K3**: 固定状态终究会丢信息，什么时候需要重新访问完整 token 历史？此外，深层网络怎样取回早期层表示？KDA 的正式公式来自 Kimi Linear 技术报告；论文明确说明它把 Gated DeltaNet 的 head-wise 标量遗忘门升级为 channel-wise 对角门控。

[Kimi Linear 论文](#)

---

## 一、回顾 Gated Delta Attention

---

下面采用 Kimi Linear 论文里的方向：

$$k_t, q_t \in \mathbb{R}^{d_k}$$

$$v_t, o_t \in \mathbb{R}^{d_v}$$

它们都是列向量，状态为：

$$S_t \in \mathbb{R}^{d_k \times d_v}$$

读取输出：

$$o_t = S_t^\top q_t$$

Gated DeltaNet 的更新公式是：

$$S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$$

其中：

- $\alpha_t \in [0, 1]$ : 全局遗忘门；
- $\beta_t \in [0, 1]$ : 当前 key 方向的擦除和写入强度。因为  $\alpha_t$  是标量，它和投影矩阵可以交换顺序：

$$\alpha_t(I - \beta_t k_t k_t^\top) = (I - \beta_t k_t k_t^\top) \alpha_t$$

因此也可以按照下面的顺序理解。

### 第一步：整体衰减

$$\bar{S}_{t-1} = \alpha_t S_{t-1}$$

### 第二步：读取当前 key 的残留旧值

$$v_t^{\text{old}} = \bar{S}_{t-1}^\top k_t$$

### 第三步：计算差值

$$u_t = \beta_t(v_t - v_t^{\text{old}})$$

### 第四步：写回状态

$$S_t = \bar{S}_{t-1} + k_t u_t^\top$$

代码：

```
# Gated DeltaNet

# alpha_t: scalar
state = alpha_t * state

old_value = state.transpose(-1, -2) @ k_t

delta = beta_t * (
    v_t - old_value
)

state = state + (
    k_t @ delta.transpose(-1, -2)
)

out = state.transpose(-1, -2) @ q_t
```

如果去掉 batch/head 维度，写成更直观的代码：

```
state = alpha * state

old = state.T @ k
delta = beta * (v - old)

state = state + torch.outer(k, delta)
out = state.T @ q
```

## 二、Gated DeltaNet 的问题：所有通道一起遗忘

假设：

$$d_k = 4$$

状态矩阵有四个 key 通道：

$$S = \begin{bmatrix} \text{身份信息} \\ \text{当前主题} \\ \text{局部语法} \\ \text{临时状态} \end{bmatrix}$$

Gated DeltaNet 产生一个标量：

$$\alpha_t = 0.8$$

那么整个状态都乘以 0.8：

$$\bar{S} = 0.8S$$

也就是：

$$\bar{S} = \begin{bmatrix} 0.8 \times \text{身份信息} \\ 0.8 \times \text{当前主题} \\ 0.8 \times \text{局部语法} \\ 0.8 \times \text{临时状态} \end{bmatrix}$$

问题在于，不同信息需要的记忆时间可能不同：

- 用户身份可能需要保留几万 token；
- 当前段落主题可能保留几百 token；
- 局部语法只需保留十几个 token；
- 已结束的临时状态应立即清除。但标量  $\alpha_t$  只能控制：

这个 head 的所有 key 通道一起衰减多少。它做不到：第 1 个通道保留 99%，第 4 个通道只保留 2%。这就是 KDA 要解决的问题。

### 三、KDA：把标量 $\alpha_t$ 变成向量

Gated DeltaNet 中：

$$\alpha_t \in \mathbb{R}$$

KDA 中：

$$\boldsymbol{\alpha}_t \in \mathbb{R}^{d_k}$$

然后构造对角矩阵：

$$D_t = \text{Diag}(\boldsymbol{\alpha}_t)$$

例如：

$$\boldsymbol{\alpha}_t = \begin{bmatrix} 0.99 \\ 0.85 \\ 0.30 \\ 0.02 \end{bmatrix}$$

那么：

$$D_t = \begin{bmatrix} 0.99 & 0 & 0 & 0 \\ 0 & 0.85 & 0 & 0 \\ 0 & 0 & 0.30 & 0 \\ 0 & 0 & 0 & 0.02 \end{bmatrix}$$

KDA 的正式状态更新为：

$$S_t = (I - \beta_t k_t k_t^\top) D_t S_{t-1} + \beta_t k_t v_t^\top$$

输出仍然是：

$$o_t = S_t^\top q_t$$

这与论文中的公式一致：

$$S_t = (I - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t) S_{t-1} + \beta_t k_t v_t^\top$$

注意：KDA 中的  $D_t$  是矩阵，不再是标量，因此一般不能随意交换顺序：

$$(I - \beta k k^\top) D \neq D (I - \beta k k^\top)$$

KDA 的实际顺序是：

1. 先用对角门  $D_t$  衰减状态；
2. 再执行当前 key 方向上的 Delta 擦除；
3. 再写入新的 value。

## 四、把 KDA 公式改写成“读一改一写”

定义经过通道衰减的状态：

$$\bar{S}_{t-1} = D_t S_{t-1}$$

然后读取当前 key 对应的旧值：

$$v_t^{\text{old}} = \bar{S}_{t-1}^\top k_t$$

计算差值：

$$u_t = \beta_t(v_t - v_t^{\text{old}})$$

写回:

$$S_t = \bar{S}_{t-1} + k_t u_t^\top$$

展开:

$$\begin{aligned} S_t &= D_t S_{t-1} + \beta_t k_t (v_t - S_{t-1}^\top D_t k_t)^\top \\ &= D_t S_{t-1} + \beta_t k_t v_t^\top - \beta_t k_t k_t^\top D_t S_{t-1} \\ &= (I - \beta_t k_t k_t^\top) D_t S_{t-1} + \beta_t k_t v_t^\top \end{aligned}$$

正好回到 KDA 公式。

## 五、从 Gated DeltaNet 到 KDA，代码只改在哪里？

### Gated DeltaNet

```
# alpha: [B, H, 1]
# 所有 key 通道共享同一个遗忘率

state = alpha.unsqueeze(-1) * state
```

状态:

```
state.shape == [B, H, Dk, Dv]
```

`alpha` 会广播到全部  $D_k \times D_v$  元素。

### KDA

```
# alpha: [B, H, Dk]
# 每个 key 通道拥有独立遗忘率
```

```
state = alpha.unsqueeze(-1) * state
```

代码看起来还是乘法，但形状发生了关键变化：

```
# Gated DeltaNet
alpha.shape == [B, H, 1]

# KDA
alpha.shape == [B, H, Dk]
```

因此 KDA 的：

```
alpha.unsqueeze(-1)
```

形状为：

```
[B, H, Dk, 1]
```

它会逐行缩放状态矩阵：

```
state 第 0 行 × alpha[0]
state 第 1 行 × alpha[1]
...
state 第 Dk-1 行 × alpha[Dk-1]
```

---

## 六、KDA 的逐 token 参考代码

```
import torch
import torch.nn.functional as F

def kda_recurrent(
```

```

q,
k,
v,
alpha,
beta,
state=None,
):
    """
    q:      [B, H, T, Dk]
    k:      [B, H, T, Dk]
    v:      [B, H, T, Dv]

    alpha: [B, H, T, Dk]
            每个 key 通道一个遗忘门

    beta:  [B, H, T, 1]
            每个 head、每个 token 一个 Delta 更新门

    state: [B, H, Dk, Dv]
    """

    q = F.normalize(
        F.silu(q),
        p=2,
        dim=-1,
    )

    k = F.normalize(
        F.silu(k),
        p=2,
        dim=-1,
    )

    B, H, T, Dk = q.shape
    Dv = v.shape[-1]

    if state is None:
        state = torch.zeros(
            B,
            H,
            Dk,
            Dv,

```



```

        device=q.device,
        dtype=q.dtype,
    )

outputs = []

for t in range(T):
    q_t = q[:, :, t]          # [B,H,Dk]
    k_t = k[:, :, t]          # [B,H,Dk]
    v_t = v[:, :, t]          # [B,H,Dv]

    alpha_t = alpha[:, :, t]  # [B,H,Dk]
    beta_t = beta[:, :, t]    # [B,H,1]

    # 1. 通道级衰减:
    # 等价于  $\text{Diag}(\alpha_t) @ \text{state}$ 
    decayed_state = (
        alpha_t.unsqueeze(-1)
        * state
    )
    # [B,H,Dk,Dv]

    # 2. 读取衰减后当前 key 方向的旧值
    old_value = (
        decayed_state.transpose(-1, -2)
        @ k_t.unsqueeze(-1)
    ).squeeze(-1)
    # [B,H,Dv]

    # 3. Delta 差值
    delta = beta_t * (
        v_t - old_value
    )
    # [B,H,Dv]

    # 4.  $k_t \text{ delta}_t^T$ 
    update = (
        k_t.unsqueeze(-1)
        @ delta.unsqueeze(-2)
    )
    # [B,H,Dk,Dv]

```

```

        state = decayed_state + update

        # 5.  $o_t = S_t^T q_t$ 
        o_t = (
            state.transpose(-1, -2)
            @ q_t.unsqueeze(-1)
        ).squeeze(-1)
        #  $[B, H, D_v]$ 

        outputs.append(o_t)

    output = torch.stack(
        outputs,
        dim=2,
    )

    return output, state

```

核心部分只有：

```

decayed_state = alpha_t.unsqueeze(-1) * state

old_value = decayed_state.transpose(-1, -2) @ k_t
delta = beta_t * (v_t - old_value)

state = decayed_state + outer(k_t, delta)
out = state.transpose(-1, -2) @ q_t

```

## 七、一个具体的通道衰减例子

假设：

$$S_{t-1} = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

两行可以理解成两个不同的 key 通道。

## Gated DeltaNet

假设：

$$\alpha_t = 0.5$$

那么：

$$\bar{S} = 0.5S_{t-1} = \begin{bmatrix} 5 & 10 \\ 15 & 20 \end{bmatrix}$$

两行都衰减 50%。

## KDA

---

假设：

$$\alpha_t = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$$

构造：

$$D_t = \begin{bmatrix} 0.9 & 0 \\ 0 & 0.1 \end{bmatrix}$$

于是：

$$\begin{aligned} \bar{S} &= D_t S_{t-1} \\ &= \begin{bmatrix} 0.9 & 0 \\ 0 & 0.1 \end{bmatrix} \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \\ &= \begin{bmatrix} 9 & 18 \\ 3 & 4 \end{bmatrix} \end{aligned}$$

结果：

- 第一通道保留 90%;
- 第二通道只保留 10%。这让同一个 attention head 内部可以同时拥有：
- 长期记忆通道;
- 中期记忆通道;
- 短期记忆通道;

- 快速清理通道。Kimi Linear 论文也把这种细粒度门控与更强的 positional awareness 联系起来：不同通道的累计衰减率不同，相当于形成不同时间尺度的位置敏感性。[Kimi Linear 论文](#)
- 

## 八、连续多个 token 后，KDA 的状态如何衰减？

---

Gated DeltaNet 的一条旧记忆经过多个 token 后，乘上标量：

$$\alpha_{i+1}\alpha_{i+2}\cdots\alpha_t$$

所有通道共享同一个保留比例。KDA 则是：

$$D_tD_{t-1}\cdots D_{i+1}$$

因为每个  $D$  都是对角矩阵，所以第  $r$  个通道的累计保留比例是：

$$\gamma_{i\rightarrow t}^{(r)} = \prod_{j=i+1}^t \alpha_j^{(r)}$$

例如两个通道：

$$\alpha^{(1)} = 0.999$$

$$\alpha^{(2)} = 0.5$$

经过 100 步后：

$$0.999^{100} \approx 0.905$$

第一通道仍保留约 90.5%。而：

$$0.5^{100} \approx 7.89 \times 10^{-31}$$

第二通道几乎完全清空。因此 KDA 可以在一个 state 里同时建立不同的时间尺度。

---

## 九、KDA 仍然解决不了什么？

---

KDA 比 Gated DeltaNet 更精细，但它仍然是固定状态模型：

$$S_t \in \mathbb{R}^{d_k \times d_v}$$

无论上下文有：

- 1K token;
- 128K token;
- 1M token; 每层保存的 recurrent state 大小基本固定。这带来高效率，但也意味着：

无限长的 token 历史最终都要被压缩进有限大小的状态。即使有更聪明的通道遗忘：

- 不同 key 仍可能互相干扰;
- 被门控衰减掉的信息无法凭空恢复;
- 精确复述早期随机字符串仍然困难;
- 固定状态无法无损保存无限多条独立关联。因此，Kimi 模型并没有把全部 Attention 都替换成 KDA。

## 十、从 KDA 到 Kimi K3：引入混合 Attention

Kimi Linear 使用的是：

```
KDA → KDA → KDA → MLA
```

即约 3:1 的混合结构。Kimi K3 延续了这个方向。官方配置是：

- 总计 93 个 Attention 层;
- 69 个 KDA;
- 24 个 Gated MLA;
- 96 个 Attention heads;
- 最大上下文长度 1,048,576。这些数字来自 Kimi K3 官方模型说明。[Kimi K3 官方仓库](#)可以把一个宏周期简化为：

第 1 层: KDA  
第 2 层: KDA  
第 3 层: KDA  
第 4 层: Gated MLA

然后重复。

## 十一、为什么采用 3 层 KDA + 1 层 MLA?

两者各自承担不同职责。

### KDA: 低成本维护压缩记忆

KDA 层保存固定状态:

$$S_t \in \mathbb{R}^{d_k \times d_v}$$

Decode 时不需要读取完整 token 历史。优点:

- 状态大小基本与上下文长度无关;
- decode 单步计算近似固定;
- 长上下文下显存和带宽压力小;
- 可以通过  $\alpha$  和  $\beta$  主动管理记忆。缺点:
- 历史被压缩;
- 可能发生关联干扰;
- 很难保证精确 token 级检索。

### MLA: 周期性访问完整 token 历史

MLA 仍然属于 softmax Attention:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)$$
$$O = AV$$

虽然它压缩了 KV 表示，但仍保留每个历史 token 的 latent KV Cache。优点：

- 可以精确访问历史 token；
- 不需要把所有信息都压进固定状态；
- 适合 needle retrieval、代码变量、数字和精确复制。缺点：
- KV Cache 仍随上下文长度增长；
- decode 时仍需要扫描历史 cache；
- 长上下文的带宽开销更高。于是 Kimi K3 采用：

多数层用 KDA 降低成本 + 少数层用 MLA 保留精确检索

Kimi Linear 报告给出的结果是，这种 3:1 混合架构在长序列生成时最多可减少约 75% KV Cache，并保留周期性的全注意力信息流。 [Kimi Linear 论文](#)

---

## 十二、Kimi K3 的 Gated MLA 怎么理解？

---

可以先看普通 MLA 的简化形式。

### 1. 压缩 latent KV

---

输入隐藏状态：

$$\boldsymbol{x}_t \in \mathbb{R}^{d_{\text{model}}}$$

先下投影：

$$\boldsymbol{c}_t^{KV} = \boldsymbol{W}_{DKV} \boldsymbol{x}_t$$

其中：

$$\boldsymbol{c}_t^{KV} \in \mathbb{R}^{d_c}$$

且：

$$d_c \ll d_{\text{model}}$$

再从 latent 表示恢复各 head 的 K、V：

$$k_t = W_{UK}c_t^{KV}$$

$$v_t = W_{UV}c_t^{KV}$$

缓存时不必保存完整 K/V，而只保存：

$$c_t^{KV}$$

## 2. Query LoRA

Kimi K3 的 query 也经过低秩路径，可以简化为：

$$c_t^Q = W_{DQ}x_t$$

$$q_t = W_{UQ}c_t^Q$$

## 3. Softmax Attention

$$a_{ti} = \text{softmax}_i \left( \frac{q_t^\top k_i}{\sqrt{d}} \right)$$

$$z_t = \sum_i a_{ti} v_i$$

## 4. 输出门

Gated MLA 再从输入产生输出 gate：

$$g_t = \sigma(W_g x_t)$$

将 Attention 输出逐元素控制：

$$y_t = g_t \odot W_O z_t$$

简化代码：

```
def gated_mla(x, latent_kv_cache):
    # Query LoRA
    q_latent = q_down_proj(x)
    q = q_up_proj(q_latent)
```



```

# 当前 token 的 Latent KV
kv_latent = kv_down_proj(x)

# Cache 保存 Latent，而不是完整 K/V
latent_kv_cache.append(kv_latent)

k = k_up_proj(latent_kv_cache)
v = v_up_proj(latent_kv_cache)

scores = q @ k.transpose(-1, -2)
scores = scores / math.sqrt(head_dim)

attn = torch.softmax(scores, dim=-1)
z = attn @ v

output = o_proj(z)

# Gated MLA
gate = torch.sigmoid(gate_proj(x))
output = gate * output

return output

```

这里的 gate 和 KDA 的  $\alpha$ 、 $\beta$  不一样：

| Gate               | 控制对象                           |
|--------------------|--------------------------------|
| KDA 的 $\alpha$     | recurrent state 中每个 key 通道保留多少 |
| KDA 的 $\beta$      | 当前关联擦除与写入多少                    |
| Gated MLA 的输出 gate | MLA 检索结果有多少进入 residual stream  |

## 十三、KDA 与 MLA 如何互补？

假设上下文中很早出现：

用户的订单号是 X7B9Q2。

经过几十万 token 后，用户问：



我的订单号是什么？

## 只用 KDA

KDA 会尝试把订单号信息编码进固定状态：

$$S_t$$

但经过很长序列后：

- 对应通道可能衰减；
- 新写入可能造成干扰；
- 精确字符序列可能无法完整保留。

## 使用 MLA

MLA 可以让当前 query 直接与早期 token 的 latent key 匹配：

$$q_{\text{订单号查询}}^T k_{\text{X7B9Q2}}$$

如果匹配分数高，就可以直接取回对应 value。因此：

- KDA 更像不断维护的“工作摘要”；
- MLA 更像可以回查的“原始档案”。Kimi K3 的策略是：

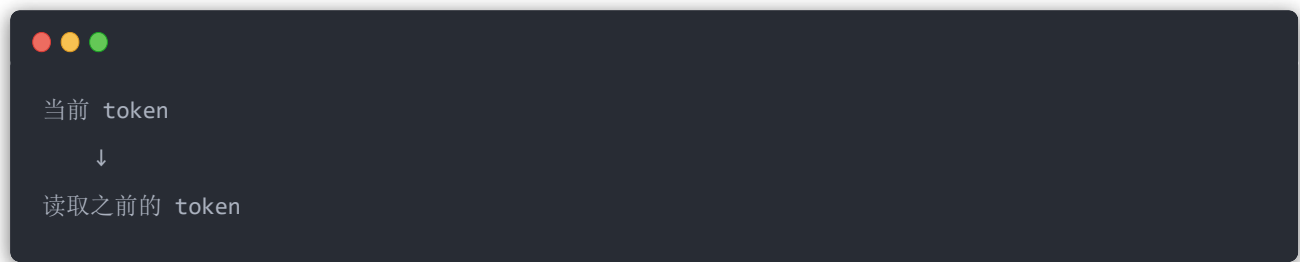
平时大部分层阅读和更新摘要，隔几层回原始档案做一次高保真检索。

## 十四、AttnRes 不是另一种 token Attention

Kimi K3 还有 AttnRes，但它和 KDA/MLA 不在同一个维度上。

## KDA 和 MLA

处理的是 token 之间的信息流：



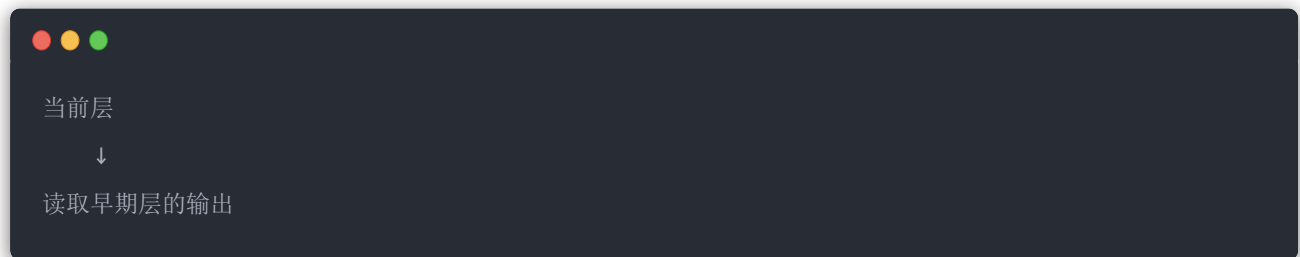
也就是沿序列长度方向：

sequence dimension

## AttnRes

---

处理的是网络层之间的信息流：



也就是沿模型深度方向：

depth dimension

Kimi K3 技术报告将两者概括为：

- KDA 改善序列长度方向的信息流；
- AttnRes 改善网络深度方向的信息流。 [Kimi K3 技术报告](#)

## 十五、普通残差连接的问题

---

普通 PreNorm Transformer 可以简化为：

$$h_{l+1} = h_l + f_l(h_l)$$

不断展开：

$$h_l = h_0 + \sum_{i=0}^{l-1} f_i(h_i)$$

也就是说，当前层输入是所有早期层输出的固定权重求和：

$$h_l = h_0 + f_0 + f_1 + \cdots + f_{l-1}$$

每一项默认权重都是 1。问题是：

- 当前层不能主动选择需要哪个早期表示；
- 层数越深，早期单层贡献越容易被稀释；
- residual stream 的幅值可能随深度增长；
- KDA 层、MLA 层和 MoE 层可能需要不同的早期特征。

---

## 十六、AttnRes：对早期层输出做 Attention

---

把早期 block 输出记作：

$$V_1, V_2, \dots, V_n$$

先归一化得到 key：

$$K_i = \text{Norm}(V_i)$$

当前层有一个学习到的 query：

$$q_l$$

计算深度方向的分值：

$$s_i = q_l^\top K_i$$

softmax 归一化：

$$a_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$$

当前层输入变成：

$$h_l = \sum_i a_i V_i$$

代码：

```
def attn_res(previous_states, query, norm):
    # [N, B, T, D]
    values = torch.stack(previous_states)

    keys = norm(values)

    # 当前层 query 与所有早期层状态匹配
    logits = torch.einsum(
        "d,nbtd->nbt",
        query,
        keys,
    )

    weights = torch.softmax(
        logits,
        dim=0,
    )

    output = torch.einsum(
        "nbt,nbtd->btd",
        weights,
        values,
    )

    return output
```

这里的 softmax 维度是：

```
dim=0
```

即在“层/块”方向做 softmax，而不是在 token 方向。AttnRes 的正式思想就是让每层通过输入相关的权重选择早期表示，而不是固定地把所有层输出等权累加。 [Attention Residuals 论文](#)

## 十七、为什么使用 Block AttnRes?

如果每一层都保留并读取全部早期层输出，成本很高。假设有 93 层，如果第 90 层要读取前面 89 个完整 hidden state:

```
89 × [B,T,D]
```

显存和跨流水线通信都会很大。因此 Kimi K3 使用 Block AttnRes:

```
若干层的输出先在 block 内累计
    ↓
形成一个 block state
    ↓
只对 block state 做深度 Attention
```

简化代码:

```
def block_attn_res(
    completed_blocks,
    current_partial_block,
    query,
    norm,
):
    values = torch.stack(
        completed_blocks
        + [current_partial_block]
    )
    # [num_blocks, B, T, D]

    keys = norm(values)

    logits = torch.einsum(
        "d,nbtd->nbt",
        query,
        keys,
    )
```

```

weights = torch.softmax(
    logits,
    dim=0,
)

h = torch.einsum(
    "nbt,nbtd->btd",
    weights,
    values,
)

return h

```

Kimi K3 中按固定层数形成残差块，可以把深度方向的候选数量从“所有层”减少为“少量 block”。

## 十八、Kimi K3 的完整 Attention 信息流

对于某个 token，在主干网络中可以简化为：





这里存在三个不同的选择机制：

| 机制               | 选择什么                            |
|------------------|---------------------------------|
| KDA 通道门 $\alpha$ | 状态的哪些通道应该长期保留                   |
| Gated MLA        | 历史 token 中哪些内容应该被精确取回，以及多少进入残差流 |
| AttnRes          | 早期网络深度中的哪些表示应该被当前层使用            |

## 十九、从 Gated DeltaNet 到 Kimi K3 的完整演化

### 1. Gated DeltaNet

$$S_t = \alpha_t(I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$$

能力：

- 全局遗忘；
- 当前 key 方向定点覆盖。 限制：
- 所有状态通道共享同一个遗忘率。

### 2. KDA

$$S_t = (I - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t) S_{t-1} + \beta_t k_t v_t^\top$$

能力：

- 每个 key 通道独立衰减；
- 同时维护多种记忆时间尺度；
- 更细粒度控制固定状态。 限制：



- 状态仍然固定大小;
- 历史仍然经过有损压缩。

### 3. Kimi K3 Hybrid Attention



69 个 KDA + 24 个 Gated MLA

能力:

- KDA 负责大多数低成本压缩记忆;
- Gated MLA 周期性恢复 token 级精确检索;
- Gated MLA 输出门控制检索结果进入残差流的比例。

### 4. AttnRes

$$h_l = \sum_i \text{softmax}_i(q_l^\top K_i) V_i$$

能力:

- 从深度方向选择早期层状态;
- 减轻深层残差稀释;
- 不再对所有早期层固定等权相加。最终可以压缩成一句话:

Gated DeltaNet: 决定整个状态忘多少

KDA: 决定状态每个通道分别忘多少

Gated MLA: 决定从哪些历史 token 精确取回什么

AttnRes: 决定从哪些早期网络层取回什么

因此, Kimi K3 的 Attention 并不是单一注意力机制, 而是两条正交的混合检索路径:

- **时间/序列方向:** KDA + Gated MLA;
- **网络深度方向:** Block AttnRes。